Git và quản trị thay đổi

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Git và lịch sử thay đổi

Git lưu lịch sử các snapshot của dự án. Working tree là file bạn đang sửa; staging area là tập thay đổi chuẩn bị đưa vào commit; commit là một mốc có định danh. Branch giúp tách luồng công việc. Remote là bản repository ở nơi khác, ví dụ máy chủ Git; **push** gửi commit lên remote, **pull** nhận thay đổi về. Git không tự quyết định rằng mọi file nên được đưa lên mạng: bạn phải kiểm tra trước khi commit, đặc biệt với secrets và file môi trường.

Quy trình tối thiểu: xem trạng thái -> sửa file -> xem diff -> stage đúng file -> commit với thông điệp rõ -> đẩy lên nhánh -> review. **git status** và **git diff** là hai lệnh nên chạy thường xuyên. Khi có conflict, Git cần bạn chọn nội dung cuối cùng; đừng xóa các dấu conflict mà không hiểu hai thay đổi đang tranh nhau điều gì.

Lab Git local

Điều kiện: đã cài Git. Trong một thư mục thử nghiệm mới, chạy **git init**, tạo **notes.txt**, rồi chạy **git status**. Thêm một dòng, chạy **git add notes.txt**, **git diff --cached**, **git commit -m "Add first note"** và **git log --oneline**. Nếu Git yêu cầu cấu hình tên/email, đọc hướng dẫn của Git trước khi cấu hình; tên và email commit có thể hiện trong lịch sử. Không dùng repository chứa dữ liệu riêng tư để thử **push**.

Mục tiêu của lab không phải thuộc lệnh. Hãy giải thích vì sao **git diff** trước stage và **git diff --cached** sau stage cho hai góc nhìn khác nhau. Thử sửa file thêm lần nữa sau commit và quan sát **git status**.

Một vòng Git có conflict được giải quyết đúng cách

Ở Giai đoạn 1, bạn cần hiểu commit và branch trước khi có CI. Tạo hai branch từ cùng một commit trong repository **thử nghiệm**. Trên branch thứ nhất, sửa dòng đầu của **notes.txt** thành "Mục tiêu: học Linux" rồi commit. Trở về branch gốc, tạo branch thứ hai từ commit ban đầu, sửa chính dòng đó thành "Mục tiêu: học networking" rồi commit. Khi merge branch thứ nhất vào branch thứ hai, Git có thể báo conflict vì hai nhánh sửa cùng vị trí. Mở file để xem phần nội dung của mỗi bên, quyết định nội dung cuối và viết lại một dòng rõ nghĩa, ví dụ "Mục tiêu: học Linux và networking". Sau đó stage và commit kết quả merge.

Điểm học không phải nhớ ký hiệu conflict. Bạn phải biết vì sao nó xuất hiện: Git không thể tự chọn giữa hai ý định. Nếu chỉ xóa ký hiệu mà không đọc hai thay đổi, có thể đánh mất nội dung. Trong team, hỏi tác giả hoặc đọc issue/PR nếu ý định không rõ. Sau merge, chạy test/đọc file để xác nhận kết quả, vì code hợp nhất có thể đúng cú pháp nhưng sai hành vi.

Tự kiểm tra Git

git status nói file nào thay đổi; **git diff** cho thay đổi chưa stage; **git diff --cached** cho thay đổi sẽ vào commit; **git log --oneline** cho lịch sử. Nếu lỡ thêm một file chứa secret vào staging nhưng chưa commit, hãy bỏ file đó khỏi staging theo tài liệu Git rồi kiểm tra lại status. Nếu secret đã được push, coi nó đã lộ và rotate; sửa lịch sử đơn thuần không đủ để bảo đảm bí mật an toàn.

Từ mã nguồn đến bản phát hành

Mã nguồn là công thức; artifact là kết quả của quy trình build. Khi một commit được đánh dấu để phát hành, ta muốn xác định chính xác mã nào, dependency nào và quy trình nào đã tạo artifact. Nếu mỗi môi trường tự build lại từ một nhánh đang thay đổi, bạn khó bảo đảm test đã chạy trên đúng thứ đang phục vụ người dùng. Cách dễ kiểm soát hơn là build một lần, định danh artifact, rồi chuyển chính artifact đó qua các môi trường.

Test giúp kiểm tra kỳ vọng, nhưng không chứng minh phần mềm không còn lỗi. Unit test cô lập hàm hoặc module; integration test kiểm tra tương tác giữa thành phần như API và database; smoke test chạy sau triển khai để xác nhận vài luồng quan trọng. Lint và static analysis bắt các vấn đề khác. Một pipeline tốt chọn lớp kiểm tra nhanh cho mỗi commit, lớp tốn kém hơn ở vị trí phù hợp, và hiển thị lỗi đủ rõ để người sửa tìm nguyên nhân.

Ví dụ cụ thể

API ghi chú có endpoint **POST /notes**. Unit test kiểm tra dữ liệu rỗng bị từ chối; integration test tạo ghi chú trong database thử nghiệm rồi đọc lại; smoke test sau deploy gọi health endpoint và một request đọc. Nếu unit test qua nhưng integration test lỗi kết nối, hướng điều tra chuyển sang database/configuration. Nếu tất cả test trước deploy qua nhưng smoke test lỗi 502, hướng điều tra chuyển sang route, proxy hoặc tiến trình vừa triển khai.

Một release có thể thất bại dù pipeline màu xanh: dữ liệu thật lớn hơn dữ liệu test, secret sai, tài nguyên thiếu hoặc dependency ngoài không đáp ứng. Vì thế cần observability và rollback từ sớm. Ghi rõ điều gì được kiểm tra tự động và điều gì chưa.

Lab trên ứng dụng mẫu

Điều kiện: ứng dụng mẫu chạy local. Viết một test cho request hợp lệ và một test cho request sai. Cố ý sửa mã để test thứ hai thất bại; xem thông báo lỗi và sửa lại. Ghi commit chứa test. Nếu chưa biết framework test của ngôn ngữ, dùng hướng dẫn chính thức của framework bạn chọn, không sao chép đoạn cấu hình không hiểu.

Git có ba vùng cần phân biệt

Working tree là file bạn đang sửa; staging index là snapshot dự kiến đưa vào commit; repository lưu commit. **git status** cho biết thay đổi ở đâu; **git diff** xem phần chưa stage; **git diff --staged** xem đúng nội dung sẽ commit. Một commit tốt mô tả một ý nghĩa thay đổi, đủ nhỏ để review/revert. Branch cho phép phát triển riêng, nhưng merge vẫn cần xem diff và test. Conflict là Git không thể tự quyết nội dung hợp nhất; chọn một phía toàn bộ không phải lúc nào đúng.

Git history còn liên quan vận hành: release phải biết commit nào tạo artifact, config hạ tầng nào áp dụng, và commit nào rollback. Một tag release là nhãn cho commit; image digest là định danh artifact. Chỉ biết branch **main** không đủ vì nhánh tiếp tục di chuyển. Với GitOps, repository cấu hình triển khai là nguồn mong muốn; quyền merge vào branch production là quyền thay đổi production.

Lab conflict có chủ đích

Trong repository lab riêng, tạo branch **feature-a** và **feature-b** từ cùng base. Sửa **cùng một dòng** cấu hình theo hai cách khác nhau, commit trên mỗi branch. Merge một branch rồi merge branch kia. Đọc conflict marker, quyết định giá trị đúng dựa trên yêu cầu của ứng dụng, sửa file, chạy kiểm tra và commit merge. Ghi **git log --graph --oneline --all** để giải thích lịch sử. Sau đó tạo một thay đổi độc lập trên nhánh mới và mở PR giả lập: mô tả mục tiêu, test, ảnh hưởng hạ tầng/dữ liệu, rollback. Không đưa **.env**, token hoặc file DB vào commit.

Revert khác reset

git revert tạo commit mới đảo tác động của commit cũ, giữ lịch sử chia sẻ; thường phù hợp nhánh đã push. **git reset** thay đổi vị trí branch/local history, dễ gây rối nếu đã chia sẻ. Nhưng revert mã nguồn chưa chắc rollback hạ tầng hay dữ liệu: Terraform plan cần review, migration DB có thể không đảo ngược. Trong phỏng vấn, khi được hỏi "rollback release", hãy chỉ rõ bạn đang rollback commit, image, manifest, config hay schema.

Branch governance và bảo vệ pipeline

Protected branch, review, required checks và quyền merge giúp giảm thay đổi không kiểm soát. Nhưng rule chỉ hiệu quả khi job CI thực sự kiểm tra artifact sẽ deploy và runner/secret không lộ cho PR không tin cậy. Một PR thay workflow để bỏ test hoặc in secret là thay đổi có rủi ro cao, cần review kỹ. Dùng lịch sử Git để truy tìm ai thay đổi gì, nhưng postmortem tập trung vào hệ thống kiểm soát và điều kiện cho lỗi lọt qua, không đổ lỗi cá nhân.

Mô hình ba vùng bằng một thay đổi thật

Git theo dõi snapshot nội dung qua commit. **Working tree** là file bạn đang sửa; **index/staging area** là snapshot dự kiến đưa vào commit; **HEAD** là commit hiện tại của branch. Giả sử bạn sửa **app.py**, sau đó **git add app.py**, rồi sửa thêm một dòng nữa. Commit kế tiếp chỉ chứa phiên bản đã stage, không tự lấy dòng sửa sau. **git diff** cho phần chưa stage; **git diff --staged** cho phần sẽ vào commit. Nếu không nắm mô hình này, rất dễ đưa nhầm secret hoặc quên file quan trọng.

Commit nên trả lời một ý định: “thêm readiness endpoint cho Notes API” thay vì “fix stuff”. Review pull request so thay đổi với base branch; người review cần thấy code, test, migration, cấu hình và rủi ro rollback. Một commit có hash là ID nội dung/lịch sử, nhưng đừng nhầm hash commit với image digest: build có thể không tái lập nếu dependency không khóa hoặc pipeline thay đổi. Ghi mối liên hệ commit -> build -> digest -> deploy revision.

Khi nào dùng merge, rebase, revert?

Merge giữ lịch sử hai nhánh và tạo merge commit khi cần; rebase phát lại commit của nhánh bạn lên base mới, làm đổi hash các commit được phát lại. Rebase nhánh cá nhân chưa chia sẻ thường dễ review; rebase lại lịch sử đã được người khác dùng có thể gây rối. Revert tạo commit mới đảo tác động của một commit trước, phù hợp lịch sử đã chia sẻ và release production. Reset di chuyển branch tip và có thể đổi index/working tree tùy chế độ; dùng thận trọng, đặc biệt **--hard** làm mất thay đổi chưa lưu. Khi chưa chắc, xem **git status**, **git log --oneline --graph**, tạo branch bảo toàn trước và chọn thao tác không mất dữ liệu.

Lab conflict có chủ đích trên repo mới

Điều kiện: Git đã cài; làm trong thư mục lab trống do bạn tạo. Không dùng repository dự án thật. Tạo **mkdir git-lab**, **cd git-lab**, **git init**, cấu hình tên/email cục bộ nếu Git yêu cầu. Tạo **config.txt** với một dòng **PORT=8000**, stage và commit. Tạo branch **feature-a**, đổi thành **PORT=8080**, commit. Quay về nhánh ban đầu (xem tên bằng **git branch --show-current**), tạo **feature-b**, đổi cùng dòng thành **PORT=9000**, commit. Merge **feature-a** vào nhánh gốc, sau đó merge **feature-b**.

Kết quả dự kiến: merge thứ hai có conflict vì cùng dòng có hai giá trị. Mở file, đọc marker **<<<<<<<**, **=======**, **>>>>>>>**. Không xóa marker rồi chọn bừa; hỏi cổng nào là quyết định sản phẩm, cấu hình có nên là biến môi trường thay vì giá trị cứng? Chọn giá trị có lý do, lưu file, **git add config.txt**, hoàn tất merge. **git status** phải sạch sau commit. Nếu merge đang dở và bạn muốn quay lại trạng thái trước merge trong repo lab, **git merge --abort** là đường phục hồi phù hợp khi còn có thể; đừng chạy trên repository chứa công việc chưa hiểu.

Thực hiện tiếp **git log --oneline --graph --all** để nhìn hai nhánh và merge commit. Ghi lại: conflict phát hiện ở **file/dòng**, không tự động chứng minh thay đổi nào đúng về nghiệp vụ. CI có thể giúp test kết quả merge, nhưng người review vẫn phải quyết định ý nghĩa. Dọn bằng cách xóa đúng thư mục lab sau khi bạn đã lưu ghi chú, không dùng lệnh reset trên dự án thật.

Branch và release governance

Với pipeline production, bảo vệ nhánh cấu hình deploy và nhánh mã nguồn. Quy định PR review, CI bắt buộc, người được merge, người được bypass, quy trình emergency và log audit. Nếu pipeline dùng secret với quyền ghi cloud, một PR thay script pipeline có thể đánh cắp secret hoặc thay hạ tầng. Chạy workflow không tin cậy với quyền hạn tối thiểu; tránh cấp token production cho job chạy code của nhánh chưa review. Signed commit/tag và provenance có thể tăng truy vết nhưng không thay review và chính sách quyền.

Tag **v1.2.3** là tên tham chiếu tới commit, không bảo đảm artifact đã được build từ commit ấy hay tag không bị di chuyển nếu policy lỏng. Release record nên gồm commit, digest, SBOM/scan hoặc bằng chứng phù hợp, deploy config revision và môi trường. Khi rollback, xác định bạn quay lại **code**, **image** hay **manifest**; DB migration có thể vẫn ở schema mới.

Ca điều tra có lời giải: hotfix không xuất hiện ở production

Developer sửa **app.py**, chạy test local, tạo commit và nói “đã push”. Production vẫn lỗi. Kiểm tra: commit có thật trên remote branch nào? PR đã merge chưa? Pipeline build commit nào? Image digest được đẩy lên registry chưa? Repo triển khai tham chiếu digest nào? Argo/CI/CD đã sync revision nào? Pod hiện chạy image nào? Mỗi câu hỏi có lệnh/UI tương ứng. Nếu developer chỉ push branch nhưng chưa merge, lỗi ở governance/delivery, không ở Kubernetes. Nếu deploy repo đã đổi nhưng Pod vẫn image cũ, xem sync/rollout. Nếu Pod đã image mới nhưng lỗi còn, hotfix chưa giải đúng nguyên nhân hoặc request đi tới bản cũ ở nơi khác.

Câu hỏi có lời giải: git revert và **git reset --hard** khác gì? Revert thêm commit đảo thay đổi và giữ lịch sử chia sẻ; reset **--hard** dời HEAD, index và working tree, có thể mất sửa đổi local. **git restore** dùng để khôi phục file ở working tree/index mà không dời branch tip. Đọc tài liệu Git chính thức trước khi dùng lệnh phá hủy lịch sử.

Reflog và cứu thay đổi trong lab

Trong repo lab, tạo hai commit A/B, sau đó thử **git reset --hard HEAD~1 chỉ sau khi xác nhận không còn thay đổi chưa commit cần giữ**. Commit B không còn trên branch, nhưng **git reflog** thường lưu tham chiếu HEAD gần đây để bạn tìm hash và tạo branch cứu B trước khi garbage collection. Mục tiêu bài này là hiểu thao tác dời branch tip và cách phòng mất dữ liệu, không khuyến khích reset lịch sử chia sẻ. Ghi hash trước/sau và vẽ branch pointer. Nếu bạn chưa chắc lệnh nào phá dữ liệu, dừng ở **git status**/**git log** và tạo bản sao lab trước.

Revert merge phức tạp hơn revert commit thường vì phải chọn mainline parent; đừng chạy mù trên production. Một hotfix release nên được xem xét cả code và config deploy: revert source commit không tự đổi image đang chạy nếu pipeline không build/promote lại. Với GitOps, revert manifest commit thường tạo desired state cũ; Argo đồng bộ lại nhưng DB/schema có thể không đảo. Vì vậy Git history là bản đồ quyết định, không phải nút thời gian quay lùi mọi hệ thống.

Review một pull request như người vận hành

PR đổi image tag trong manifest và thêm env **FEATURE_X=true**. Hãy hỏi: image digest đã build/test chưa, tag có bất biến không, env ảnh hưởng DB/traffic ra sao, rollback có tương thích dữ liệu không, **resources**/probe có đổi không, secret nào được tham chiếu, staging có chạy cùng artifact không. Nếu PR cũng sửa pipeline, xem quyền runner/secret và trigger. Một diff 2 dòng có thể có blast radius lớn hơn 500 dòng tài liệu.

Bài tập: tạo PR giả trên giấy với thay đổi **replicas: 2 -> 4** và **maxConnections: 100 -> 400**. App tăng Pod nhưng DB max connections có thể không đủ, hoặc connection storm làm DB nghẹt. Viết review comment hỏi capacity DB, HPA, load test và kế hoạch quan sát. Đây là “Git governance” nối với kiến trúc chứ không chỉ check style.

Git tags, branching và hotfix

Trunk-based development giữ branch sống ngắn, tích hợp thường xuyên; release branch dài hơn có thể cần khi hỗ trợ nhiều version nhưng tăng công việc backport. Không có một chiến lược thắng mọi đội; chọn theo nhịp phát hành, quy mô, quy định và khả năng test. Một hotfix production cần commit trên đúng base, test, review khẩn cấp, build artifact, promote, rồi merge/backport lại các nhánh khác để lỗi không quay lại. Nếu sửa trực tiếp nhánh production mà quên main, lần release sau có thể mất hotfix.

Câu hỏi có lời giải: **git pull** báo conflict, dùng **git push --force** được không? Force push không giải conflict và có thể ghi đè lịch sử người khác. Đọc branch/remote, fetch, giải quyết merge/rebase trong nhánh của bạn và chỉ force-with-lease khi chính sách cho phép trên branch cá nhân đã rebase. **Secret commit nhầm rồi xóa ở commit sau có đủ không?** Không; token vẫn trong lịch sử/caches, phải revoke/rotate, hạn chế access và xử lý lịch sử theo quy trình.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://git-scm.com/docs
- https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository